TEORIE

Arhitectura Client-Server:

- Relațiile dintre Subsistemele Arhitecturii
  - Clientii cunosc interfata serverului.
  - Serverul nu trebuie sa cunoasca interfetele clientilor
  - La executie, clientul si serverul sunt procese independente care ruleaza, de regula, pe calculatoare diferite, sincronizarea lor fiind necesara numai pentru ca serverul sa raspunda la cereri si clientul sa primeasca rezultate.
- Rolul Fiecărui Subsistem
  - Clientii: - primesc intrari de la utilizatori, efectueaza verificarea datelor introduse si apeleaza servicii ale serverului.
  - Serverul: - raspunde la cererile clientului efectuand tranzactii cu baza de date
    - asigura controlul accesului concurent la baza de date
    - asigura integritatea si securitatea datelor
    - asigura recuperarea datelor in cazul caderilor
- Criterii de Proiectare a Aplicațiilor cu Arhitectura Client-Server
  - Portabilitate: - Clientul sa poata rula pe diverse dispozitive si sisteme de operare - Serverul sa poata fi accesat din orice sistem de operare şi independent de mediul de comunicare
  - Transparenţa locatiei: clientul nu trebuie sa cunoasca locatia/locatiile serverului
  - Performanţă înaltă: • Client optimizat pentru taskuri de interactiune (actualizarea rapida a informatiei afisate, primita de la server)
    • Serverul optimizat pentru operatii CPU – intensive/ operatii cu baze de date
  - Scalabilitate: serverul sa poata trata un numar mare de clienti
  - Fiabilitate

Arhitectura Three-tier:

- Caracteristici
  - Arh ierarhică închisă, în 3 niveluri, de regula alocate pe noduri hardware dist
  - Specifică sistemelor care includ o baza de date.
  - **interfaţă utilizator** (Presentation tier) – responsabil cu interactiunea prin ferestre, form-uri, pagini web
  - **aplicaţie** (Application logic/ Logic tier) = **Middleware** – include prelucrarile specifice aplicatiei si comunicarea dintre nivelul interfaţă şi nivelul stocare
  - **stocare** (Storage/ Data tier) – sistemul de gestiune a bazei de date – asigura stocarea si regasirea obiectelor persistente.
- Utilizari

  Arhitectura este des folosită în sistemele bazate pe Web:
  - **interfaţă utilizator** – “frontend” - implementat folosind un framework de dezvoltare web (ex. JavaScript, CSS) - se executa într-un browser care ruleaza pe dispozitivul utilizatorului
  - **aplicatie** – “backend” - se executa pe un **server web**
    - **browser**-ul asigura comunicatia dintre frontend si backend
  - **stocare** - se executa pe un server de baze de date - datele persistente sunt gestionate de un sistem de gestiune al bazei de date (MongoDB, Oracle, MySQL, Microsoft SQL Server)

- Avantaje
    - Cele 3 niveluri **pot fi dezvoltate** în mod independent
    - **Fiecare n poate fi îmbunatatit** sau **înlocuit independent**, în cazul unei schimbari a cerintelor sau a unei schimbari tehnologice (hardware nou).
    - Nivelul **stocare**, analog unui subsistem Repository, poate fi partajat de diferite aplicatii care utilizeaza aceeasi baza de date.
    - **Separarea** nivelului **interfata** de nivelul **aplicatie** permite existenta mai multor interfețe utilizator pentru subsistemul/subsistemele care implementeaza logica aplicatiei.

Repository - centrat pe date

- Caracteristici
    - Repository, responsabil cu persistenta datelor si accesul la depozitul de date.
    - Mai multi „accesori” - comunica intre ele numai prin Repository pentru a accesa depozitul de date si a modifica datele din depozit
    - Scopul arhitecturii: asigurarea integritatii datelor
    - In functie de fluxul controlului avem **pasiv** sau **activ**

PASIV

- Caracteristici
    - Repository pastreaza o structura de date centrala, pasiva
    - Subsistemele accesor sunt relativ independente si activi– controleaza fluxul prelucrarilor.
    - pentru sistemele care folosesc baze de date, compilatoare si sisteme de dezvoltare software
- Avantaje si dezavantaje
    - Asigura integritatea datelor, operatii de backup si restaurare a datelor
    - Este scalabilă – pot exista oricâți accesori
    - Reduce overhead-ul produs de transmisia datelor între componentele software
    - Accesul poate conduce la o “strangulare”→ scaderea performantei;
    - Cuplarea intre Repository si fiecare alt subsistem este mare →modificarea Repository-ului afecteaza toate celelalte subsisteme.

ACTIV

- Caracteristici
    - Fluxul prelucrarilor este determinat de starea curenta a depozitului de date.
    - Componenta Repository notifica toti accesorii atunci cand sunt modificate datele din depozit.
- Avantaje si dezavantaje
    - Accesorii se executa in paralel, fiind complet independenti intre ei
    - Scalabilitate: pot fi adaugati oricati accesori
    - Reutilizabilitatea codului accesorilor
    - Dependenta puternica intre componente: modificarile structurale ale depozitului pot afecta toti accesorii (GetState() intoarce starea depozitului)
    - Probleme in sincronizarea accesorilor → dificultati in proiectare si testare

Testare de sistem

- Scopul

    - Se verifică dacă produsul obtinut (hardware+software) satisface specificația cerințelor
- Cine o efectueaza
    - Este efectuată de o echipă a companiei dezvoltatoare.
- Tipuri teste
    - derivate din cerințele funcționale – teste funcționale
    - derivate din cerințele nefuncționale - teste de performanţă, fiabilitate, portabilitate, disponibilitate, securitate, siguranta in functionare, s.a.

Testare de acceptare

- Scopul
    - verificarea conformitatii produsului obtinut cu produsul solicitat, conform contractului cu clientul
- Cine o efectueaza
    - Clientul şi utilizatorii finali participă la efectuarea acestor teste
- Tipuri teste
    - **Functionalitatea**: produsul trebuie sa ofere toate funcțiile prevazute în contract şi să le execute corect
    - **Acuratetea**: precizia rezultatelor
    - **Integritatea datelor:** se verifica daca datele dintr-o baza de date nu sunt alterate prin operatii de interogare, actualizare, restaurare
    - **Recuperarea datelor**
    - **Fiabilitatea:** rata caderilor in timpul testelor de acceptare

Testare bazata pe clase de echivalenta

- Ce este?
    - Foloseste o partiționare a vectorilor de test in clase de echivalență.
    - O **clasă de echivalență** a domeniului de intrare al unității testate este alcatuită din **toți vectorii de test** care produc aceeaşi comportare a unității testate (funcție/program).
- Cum se aleg cazurile pt min/ min robusta
    - Pentru testarea **minimală** este suficient sa se foloseasca un singur caz de test pentru fiecare clasă de echivalență → Numarul de cazuri de test este mai mic decat în cazul folosirii tuturor combinatiilor între valorile selectate din domeniile variabilelor de intrare.
    - Pentru testarea **minimală robusta** se adaugă cazuri de test cu valori din afara claselor de echivalență ale DI al programului:

Testare de integrare

- Obiective
    - integrarea unitatilor program (module program: unitati functionale/clase) dezvoltate si testate independent, într-un sistem functional.
- Cum se efectueaza
    - Se verifica interactiunile dintre module, grupuri de module, subsisteme, pana la nivel de sistem.
    - Clasele au fost testate independent. Prin testele de integrare se verifica interactiunea dintre obiectele claselor prin suite de test.
    - Testele de integrare presupun, ca si testele unitare, realizarea de module "ciot" si module "driver ", pentru a simula modulele **neintegrate**.
    - Numarul de module "driver " şi "ciot" necesare in testele de integrare depinde de ordinea in care sunt integrate modulele.

    - Testele de integrare necesita **instrumente de gestiune a versiunilor si a configuratiilor**

Principiile generale ale dezvoltarii agile

- Clientul trebuie să fie puternic implicat în procesul de dezvoltare. Rolul lui este de a furniza şi prioritiza cerinţele şi de a evalua iteraţiile dezvoltării.
- Produsul software este dezvoltat în incremente, clientul specificand cerinţele care trebuie sa fie implementate în fiecare increment. Metodologii şi cadre de dezvoltare agilă
- Abilitaţile echipei de dezvoltare trebuie sa fie recunoscute şi exploatate. Membrii echipei trebuie lasaţi să-şi dezvolte propriul stil de lucru, fără să li se impuna procese prescriptive.
- Schimbarea cerinţelor este acceptată pe tot parcursul dezvoltării, de aceea sistemul trebuie proiectat pentru a fi uşor adaptat la schimbari.
- Focalizare pe simplitate atât în produsul software dezvoltat cât şi în procesul de dezvoltare. Elimină complexitatea oricând este posibil.

Deosebire: Fiabilitate vs Robustete

- **Fiabilitatea (Reliability)**: Este definită ca capacitatea sistemului de a funcționa conform comportamentului specificat. Cu alte cuvinte, fiabilitatea se referă la măsura în care un sistem software execută funcțiile sale fără erori în condiții specificate și pe o perioadă de timp determinată .
- **Robustețea (Robustness)**: Este capacitatea sistemului de a rezista și a continua să funcționeze corect în prezența intrărilor neprevăzute sau în condiții anormale. Robustețea se referă la reziliența sistemului față de situațiile neanticipate care ar putea apărea în mediul de operare .

Aceste definiții evidențiază că fiabilitatea este axată pe performanța corectă în condiții normale și specificate, în timp ce robustețea se concentrează pe capacitatea de a face față și a continua să funcționeze în fața evenimentelor neprevăzute și a condițiilor anormale.

Relaţii între cazurile de utilizare

- Includere – un caz de utilizare A include, în unul sau mai multi pasi, comportamentul descris într-un alt caz de utilizare B

- Extindere – un caz de utilizare E extinde un alt caz de utilizare, A, daca A include comportamentul descris in E, numai in anumite conditii

- Generalizare/specializare: similara conceptual cu relatia de generalizare dintre clase B este o specializare a cazului de utilizare A

Dezvoltare incrementala vs dezvoltare liniara(1 sg ciclu)

- Avantaje

    - In fiecare iteratie se obtine un produs executabil, care satisface o parte din cerintele utilizator.
    - Produsul este mai fiabil decat intr-o dezvoltare in cascada → aspectele cele mai importante ale produsului sunt cel mai mult testate.
- Riscuri reduse
    - se poate transforma usor in ‘construieste si repara’ (build and fix)→ afecteaza calitatea produsului final: numarul de defecte ramase este mare, intretinerea dificila
    - costul este dificil de estimat

Dezvoltarea liniară (într-un singur ciclu de dezvoltare): - Modelul cascadă (Waterfall), modelul în V, dezvoltarea pe baza de prototip.
Dezvoltarea iterativă şi incrementală (în mai multe cicluri de dezvoltare): - RUP (Rational Unified Process), modele agile (SCRUM, XP, AUP şi altele).

Dezvoltarea cascada

- Avantaje
    - Produsul software este bine documentat.
    - Permite un bun management al proiectului:
      –planificarea resurselor umane pe etape
      –estimari de cost destul de exacte
      –clientul este implicat numai la începutul procesului de dezvoltare
- Dezavantaje
    - Un produs executabil, care sa demonstreze functionarea sistemului este disponibil tarziu → risc crescut de neacceptare a produsului
    - Multe defecte sunt descoperite tarziu; in special cele de proiectare → cost crescut (sunt reluate activitatile de proiectare, implementare, testare).
    - Inflexibil la modificarea cerintelor pe parcursul dezvoltarii (cerintele sunt definite la inceputul procesului si cu greu se accepta modificarea lor pe parcurs)
    - Toate riscurile sunt incluse intr-un singur ciclu de dezvoltare

Dezvoltarea agila

- Avantaje
    - **Focalizate pe adaptarea rapidă la schimbări. Nu se planifică ce se va întâmpla în viitor.**
    - Perioadele iterațiilor sunt scurte şi fixe.
    - Livrări frecvente.
    - **Implicare puternică a clientului pe parcursul dezvoltarii.**
    - Echipa lucrează într-o manieră colaborativă, prin comunicare directă.
    - Documentație minimală.
    - **Rezultatul final este decis printr-un proces de negociere pe parcursul dezvoltării.**
- Dezavantaje
    - **Predictibilitate scazută** - Pentru anumite livrabile, dezvoltatorul nu poate cuantifica exact efortul necesar, în special la începutul proiectelor mari, ceea ce crează frustrări echipelor începătoare în dezvoltarea agilă.
    - **Mai mult timp şi angajare**

- Dezvoltatorii, testerii şi clientul trebuie sa interacționeze direct (face to face) în mod constant pe parcursul dezvoltării.

- Utilizatorii trebuie sa fie disponibili pentru testarea rapidă, înainte de a se trece la următoarea etapă a dezvoltării.
- Clientul trebuie sa fie instruit pentru a ajuta în dezvoltarea produsului. Lipsa de participare a clientului impacteaza calitatea produsului şi succesul proiectului.

- **Lipsa documentației necesare**

- Documentația este minimală. Membrii noi în echipă nu cunosc detaliile anumitor functionalități şi cum trebuie realizate, ceea ce crează neînțelegeri şi dificultăți.

- **Proiectul poate cu uşurință să-şi piardă ținta**

- Dezvoltarea agilă porneşte cu planificare minimală şi presupune că cerințele clientului se vor schimba continuu. Dacă feedback-ul utilizatorilor sau comunicarea nu este clară, dezvoltatorul se poate focaliza pe o direcție de dezvoltare greşită

Dezvoltare dirijata de teste (TDD)

- Desfasurare
  - Test driven development = Test first development + Refactoring
  - **Test-first development:**

▪ Programatorul scrie testele unitare înainte de a scrie codul testat
▪ Testele sunt dezvoltate incremental pornind de la scenarii.
▪ Fiecare test corespunde unei cerințe.
▪ Reprezentantul utilizatorilor/clientului este implicat în dezvoltarea şi validarea testelor.
▪ Sunt folosite instrumente de testare unitară automată.
▪ Atunci când este adăugat/modificat cod sunt rulate toate testele, atât cele anterioare cât şi testul pentru codul adăugat/modificat – se verifică astfel ca noul cod nu a introdus erori.

- **Refactorizare (îmbunătățirea codului)**

▪ Rescrierea codului, fără a modifica funcționalitatea sa externă, pentru îmbunătățirea unor atribute ale codului, cum ar fi: eficiența, claritatea, complexitatea, uşurința de întreținere.
➢ Cod uşor de înțeles → reduce necesitatea documentației.
➢ Cod bine structurat şi clar → uşurează efectuarea modificărilor

- Beneficii
  - Conduce la dezvoltarea de software cu grad înalt de utilizabilitate (usability): uşurința de înțelegere, de învățare, de operare, plăcerea de a utiliza produsul, de către utilizatorii țintă
  - Reduce numărul de defecte din cod: sunt eliminate in mare parte prin testele unitare
  - Evidențiază problemele de funcționalitate în fazele incipiente → reduce costul rezolvarii problemelor de implementare a funcționalitatilor cerute.
  - Simplifică codul
  - Conduce la cod extensibil: dezvoltatorul este încurajat să construiască software alcătuit din unități mici de cod testabil.
  - Permite obtinerea de feedback rapid de la client/utilizatori.
  - Documentarea actualizată a codului: testele sunt exemple de utilizare a codului.

GRILE
Modelul de dezvoltare "In cascada"

a.Este adecvat proiectelor cu perioadă lungă de dezvoltare
b.Nu permite beneficiarului să utilizeze versiuni intermediare ale produsului software
c.Este adecvat proiectelor în care cerințele se pot schimba pe parcursul dezvoltării
d.Permite un bun management al proiectului

Care dintre următoarele afirmații sunt adevărate relativ la modelul de dezvoltare "In V"?
a.Incurajează implicarea clientului în procesul de dezvoltare.
b.Produsul software este dezvoltat într-un singur ciclu de dezvoltare.
c.Este un model de dezvoltare iterativă.
d.Planurile de test sunt realizate în etapele de dezvoltare anterioare codificării.

O diagrama de colaborare

a.Reda schimbul de mesaje intre obiecte
b.Nu poate reda ordinea mesajelor schimbate intre obiecte
c.Nu poate reda trimiterea conditionata a mesajelor
d.Contine obiecte si legaturi intre obiecte

Un actor este:
a.O persoana, sistem sau echipament care interactioneaza cu sistemul.
b.O persoana care primeste mesaje de la sistem sau un echipament de iesire.
c.O persoana care raspunde de sistem.

Un scenariu este:
a.Un set de optiuni de navigare pe ecran
b.O singura cale logica intr-un caz de utilizare
c.O cale de executie in sistem
d.O combinatie de cazuri de utilizare

Diagramele de secventa se folosesc pentru
a.Reprezentarea interfetei utilizator
b.Modelarea scenariilor
c.Redarea schimbului de mesaje intre obiecte in ordinea temporala
d.Modelarea interactiunii utilizator-sistem

Cazurile de utilizare se folosesc pentru:
a.Testarea sistemului
b.Modelarea structurala a unui sistem
c.Modelarea comportamentala a unui sistem
d.Analiza functionala

Modelul de dezvoltare in spirala:
a.Este un model de dezvoltare iterativa si incrementala

b.Include analiza si rezolvarea riscurilor in fiecare iteratie
c.Este adecvat oricarui tip de proiect

d.Permite o buna planificare in timp a proiectului

O diagrama de colaborare
a.Reda schimbul de mesaje intre obiecte
b.Contine obiecte si legaturi intre obiecte
c.Nu poate reda trimiterea conditionata a mesajelor
d.Nu poate reda ordinea mesajelor schimbate intre obiecte